LAB report TCP server to Go

Link to GitHub: https://github.com/KaiserAAAAAAAAAAN/LAB9-Build-and-Analyze-a-Client-Server-Socket-System

Cause of crashing:

Main reason the server crash is because of concurrency and resource exhaustion, not really from the network problem. When many clients connect at a lot of goroutines and keep many connections open. This increases CPU and memory usage and makes the system harder to manage.

Debugging evident.

```
18:20:44.910818 [DISCONNECT] 127.0.0.1:62962 | EOF
18:20:44.910818 [DISCONNECT] 127.0.0.1:62963 | EOF
18:20:44.910818 [RECV] 127.0.0.1:62992 → "Hello from client 59"
18:20:44.910818 [DISCONNECT] 127.0.0.1:62964 | EOF
18:20:44.911342 [RECV] 127.0.0.1:62994 → "Hello from client 72"
18:20:44.911342 [DISCONNECT] 127.0.0.1:62965 | EOF
18:20:44.911342 [RECV] 127.0.0.1:62995 → "Hello from client 87"
18:20:44.911342 [RECV] 127.0.0.1:62997 → "Hello from client 90"
18:20:44.911342 [DISCONNECT] 127.0.0.1:62966 | EOF
18:20:44.911867 [DISCONNECT] 127.0.0.1:62967 | EOF
18:20:44.911867 [RECV] 127.0.0.1:62998 → "Hello from client 91"
18:20:44.911867 [DISCONNECT] 127.0.0.1:62916 | EOF
18:20:44.911867 [DISCONNECT] 127.0.0.1:62969 | EOF
18:20:44.911867 [RECV] 127.0.0.1:63000 → "Hello from client 94"
18:20:44.911867 [RECV] 127.0.0.1:63002 → "Hello from client 28"
18:20:44.911867 [RECV] 127.0.0.1:62996 → "Hello from client 27"
18:20:44.912388 [DISCONNECT] 127.0.0.1:62968 | EOF
18:20:44.912388 [RECV] 127.0.0.1:63003 → "Hello from client 88"
18:20:44.912388 [RECV] 127.0.0.1:63005 → "Hello from client 89"
18:20:44.912388 [RECV] 127.0.0.1:63004 → "Hello from client 75"
18:20:44.912388 [DISCONNECT] 127.0.0.1:62917 | EOF
18:20:44.912388 [RECV] 127.0.0.1:63001 → "Hello from client 74"
18:20:44.912388 [RECV] 127.0.0.1:62999 → "Hello from client 24"
18:20:44.912388 [DISCONNECT] 127.0.0.1:62970 | EOF
18:20:44.912388 [RECV] 127.0.0.1:63008 → "Hello from client 93"
18:20:44.912911 [RECV] 127.0.0.1:63006 → "Hello from client 58"
18:20:44.912911 [RECV] 127.0.0.1:63007 → "Hello from client 60"
18:20:44.912911 [DISCONNECT] 127.0.0.1:62972 | EOF
18:20:44.912911 [RECV] 127.0.0.1:63009 → "Hello from client 73"
18:20:44.912911 [RECV] 127.0.0.1:63011 → "Hello from client 97"
18:20:44.912911 [DISCONNECT] 127.0.0.1:62973 | EOF
18:20:44.912911 [RECV] 127.0.0.1:63010 → "Hello from client 92"
18:20:44.912911 [DISCONNECT] 127.0.0.1:62974 | EOF
18:20:44.913432 [RECV] 127.0.0.1:63012 → "Hello from client 95"
18:20:44.913432 [RECV] 127.0.0.1:63013 → "Hello from client 98"
18:20:44.913432 [DISCONNECT] 127.0.0.1:62975 | EOF
18:20:44.913432 [RECV] 127.0.0.1:63014 → "Hello from client 99"
18:20:44.913432 [DISCONNECT] 127.0.0.1:62976 | EOF
18:20:44.913432 [RECV] 127.0.0.1:63015 → "Hello from client 96"
```

From the server logs during load testing, we can clearly see that many clients are connecting at the same time. For example:

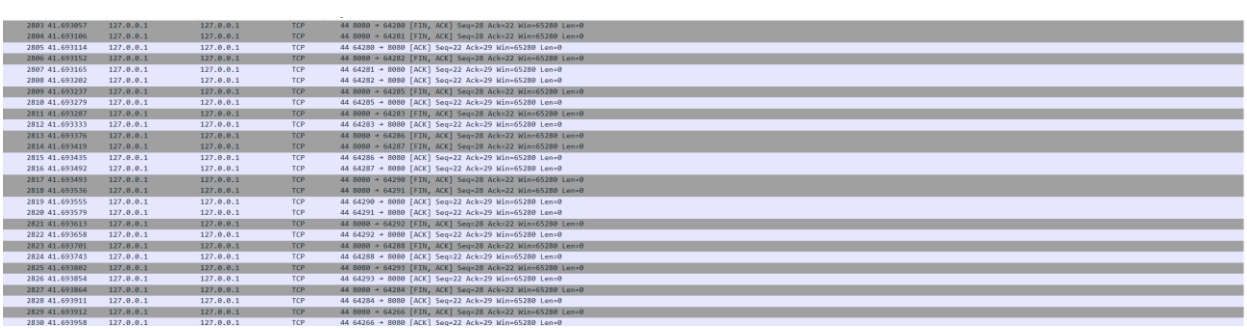
```
18:20:44.910818 [DISCONNECT] 127.0.0.1:62962 | EOF
18:20:44.910818 [DISCONNECT] 127.0.0.1:62963 | EOF
18:20:44.910818 [RECV] 127.0.0.1:62992 → "Hello from client 59"
18:20:44.910818 [DISCONNECT] 127.0.0.1:62964 | EOF
18:20:44.911342 [RECV] 127.0.0.1:62994 → "Hello from client 72"
18:20:44.911342 [DISCONNECT] 127.0.0.1:62965 | EOF
18:20:44.911342 [RECV] 127.0.0.1:62995 → "Hello from client 87"
18:20:44.911342 [RECV] 127.0.0.1:62997 → "Hello from client 90"
18:20:44.911342 [DISCONNECT] 127.0.0.1:62966 | EOF
18:20:44.911867 [DISCONNECT] 127.0.0.1:62967 | EOF
18:20:44.911867 [RECV] 127.0.0.1:62998 → "Hello from client 91"
18:20:44.911867 [DISCONNECT] 127.0.0.1:62916 | EOF
18:20:44.911867 [DISCONNECT] 127.0.0.1:62969 | EOF
18:20:44.911867 [RECV] 127.0.0.1:63000 → "Hello from client 94"
18:20:44.911867 [RECV] 127.0.0.1:63002 → "Hello from client 28"
18:20:44.911867 [RECV] 127.0.0.1:62996 → "Hello from client 27"
18:20:44.912388 [DISCONNECT] 127.0.0.1:62968 | EOF
18:20:44.912388 [RECV] 127.0.0.1:63003 → "Hello from client 88"
18:20:44.912388 [RECV] 127.0.0.1:63005 → "Hello from client 89"
18:20:44.912388 [RECV] 127.0.0.1:63004 → "Hello from client 75"
18:20:44.912388 [DISCONNECT] 127.0.0.1:62917 | EOF
18:20:44.912388 [RECV] 127.0.0.1:63001 → "Hello from client 74"
18:20:44.912388 [RECV] 127.0.0.1:62999 → "Hello from client 24"
18:20:44.912388 [DISCONNECT] 127.0.0.1:62970 | EOF
18:20:44.912388 [RECV] 127.0.0.1:63008 → "Hello from client 93"
18:20:44.912911 [RECV] 127.0.0.1:63006 → "Hello from client 58"
18:20:44.912911 [RECV] 127.0.0.1:63007 → "Hello from client 60"
18:20:44.912911 [DISCONNECT] 127.0.0.1:62972 | EOF
18:20:44.912911 [RECV] 127.0.0.1:63009 → "Hello from client 73"
18:20:44.912911 [RECV] 127.0.0.1:63011 → "Hello from client 97"
18:20:44.912911 [DISCONNECT] 127.0.0.1:62973 | EOF
18:20:44.912911 [RECV] 127.0.0.1:63010 → "Hello from client 92"
18:20:44.912911 [DISCONNECT] 127.0.0.1:62974 | EOF
18:20:44.913432 [RECV] 127.0.0.1:63012 → "Hello from client 95"
18:20:44.913432 [RECV] 127.0.0.1:63013 → "Hello from client 98"
18:20:44.913432 [DISCONNECT] 127.0.0.1:62975 | EOF
18:20:44.913432 [RECV] 127.0.0.1:63014 → "Hello from client 99"
18:20:44.913432 [DISCONNECT] 127.0.0.1:62976 | EOF
18:20:44.913432 [RECV] 127.0.0.1:63015 → "Hello from client 96"
```

As a result

This shows that:

- many clients are connected simultaneously
- active connections increase rapidly (up to around 100)
- the server is handling multiple requests at the same time This confirms that the server is under high concurrency, which increases resource usage and can lead to performance issues or crashes.

Wireshark



2803	41.693057	127.0.0.1	127.0.0.1	TCP	44 8080 → 64280 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2804	41.693106	127.0.0.1	127.0.0.1	TCP	44 8080 → 64281 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2805	41.693114	127.0.0.1	127.0.0.1	TCP	44 64280 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2806	41.693152	127.0.0.1	127.0.0.1	TCP	44 8080 → 64282 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2807	41.693165	127.0.0.1	127.0.0.1	TCP	44 64281 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2808	41.693202	127.0.0.1	127.0.0.1	TCP	44 64282 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2809	41.693237	127.0.0.1	127.0.0.1	TCP	44 8080 → 64285 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2810	41.693279	127.0.0.1	127.0.0.1	TCP	44 64285 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2811	41.693287	127.0.0.1	127.0.0.1	TCP	44 8080 → 64283 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2812	41.693333	127.0.0.1	127.0.0.1	TCP	44 64283 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2813	41.693376	127.0.0.1	127.0.0.1	TCP	44 8080 → 64286 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2814	41.693419	127.0.0.1	127.0.0.1	TCP	44 8080 → 64287 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2815	41.693435	127.0.0.1	127.0.0.1	TCP	44 64286 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2816	41.693492	127.0.0.1	127.0.0.1	TCP	44 64287 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2817	41.693493	127.0.0.1	127.0.0.1	TCP	44 8080 → 64290 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2818	41.693536	127.0.0.1	127.0.0.1	TCP	44 8080 → 64291 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2819	41.693555	127.0.0.1	127.0.0.1	TCP	44 64290 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2820	41.693579	127.0.0.1	127.0.0.1	TCP	44 64291 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2821	41.693613	127.0.0.1	127.0.0.1	TCP	44 8080 → 64292 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2822	41.693658	127.0.0.1	127.0.0.1	TCP	44 64292 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2823	41.693701	127.0.0.1	127.0.0.1	TCP	44 8080 → 64288 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2824	41.693743	127.0.0.1	127.0.0.1	TCP	44 64288 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2825	41.693802	127.0.0.1	127.0.0.1	TCP	44 8080 → 64293 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2826	41.693854	127.0.0.1	127.0.0.1	TCP	44 64293 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2827	41.693864	127.0.0.1	127.0.0.1	TCP	44 8080 → 64284 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2828	41.693911	127.0.0.1	127.0.0.1	TCP	44 64284 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0
2829	41.693912	127.0.0.1	127.0.0.1	TCP	44 8080 → 64266 [FIN, ACK] Seq=28 Ack=22 Win=65280 Len=0
2830	41.693958	127.0.0.1	127.0.0.1	TCP	44 64266 → 8080 [ACK] Seq=22 Ack=29 Win=65280 Len=0

Wireshark Tooling

The network analysis was performed using the following filter:
tcp.port == 8080

Observations
From the captured packets, it is evident that numerous TCP connections are occurring near-simultaneously. Multiple **SYN packets** are initiated from various source ports to port 8080, followed by the standard **SYN-ACK** and **ACK** sequence. This confirms that TCP handshakes are executing in parallel.

Technical Implications

- **Concurrent Handling:** The server is managing multiple unique client connections simultaneously, each assigned a distinct source port.
- **Resource Demand:** The high volume of SYN packets indicates a heavy connection load, which significantly increases server workload and resource consumption.
- **Root Cause:** The Wireshark results confirm the primary issue is **high concurrency** and connection volume rather than a localized network failure.

3. Metrics

To maintain system health, the following key performance indicators (KPIs) must be monitored:

1. **Active Connections:** Tracks the total number of simultaneous client sessions.
2. **Goroutine Count:** Measures the number of concurrent tasks currently active within the system.
3. **Error/Timeout Rate:** Monitors the frequency of failed connections or request timeouts.
   Note: Supplemental metrics such as CPU usage, memory saturation, and response latency should also be tracked.

4. Mitigation Strategy

To stabilize the environment and manage load, the following solutions have been implemented:

- **Connection Limiting:** Utilizing a channel-based semaphore to cap the maximum number of simultaneous client connections.
- **Timeouts:** Implementing strict deadlines for each connection to prevent "slow-loris" style resource exhaustion.
- **Input Validation:** Enforcing strict checks on input size to reject oversized or malicious data packets.
- **Enhanced Logging:** Recording detailed connection activity to improve debugging and real-time monitoring.

5. Conclusion

The system instability is primarily driven by **concurrency bottlenecks** and **resource management issues** rather than external network faults. Analysis of logs and stress testing confirms that connection and goroutine counts spike significantly under load. By implementing connection limits, timeouts, and validation protocols, the server can maintain stability under high demand.